

High-Concurrency Booking Engine — Submission

Airtribe Backend System Design — *High-Concurrency Booking Engine*

P1 — Entities, Tables, and Locking Strategy

Entity overview

Table	Purpose
city, theatre, screen	Location hierarchy: a city has theatres, a theatre has screens
seat_category	Silver / Gold / Recliner tiers
seat	One row per physical seat in a screen (row, number, category)
language, show_format, movie	Catalog/reference data
movie_show	One scheduled screening: screen + movie + language + format + date + time
show_price	Price per (show, seat category)
app_user	Registered users
seat_hold	A user's temporary claim on N seats of one show, with an expiry
show_seat	The concurrency table — one row per (show, physical seat), tracking AVAILABLE / HELD / BOOKED
booking	A confirmed booking created from a hold
payment	Payment order for a booking
payment_webhook_event	Idempotency ledger for payment gateway webhooks

Full DDL: `sql/01_schema.sql`. Sample data: `sql/02_seed.sql`.

Normalization (1NF → BCNF)

- **1NF**: every column is atomic (no comma-lists); e.g. seats are individual rows in `seat/show_seat`, not a JSON blob on `movie_show`.
- **2NF**: every non-key column depends on the *whole* key. `show_price` has a composite key (`show_id`, `category_id`) and price depends on both, not a subset — so it's its own table rather than a column bolted onto `movie_show`.
- **3NF**: no transitive dependencies. `theatre_id` is **not** repeated on `screen`, `movie_show`, or `show_seat` — it's derived by joining through `screen`, so a theatre rename happens in exactly one row.
- **BCNF**: every determinant is a candidate key. `movie_show` has `UNIQUE(screen_id, show_date, start_time)` (a screen can't run two shows at once) *and* the surrogate `show_id` as PK — both are keys, so there's no partial dependency hiding a BCNF violation.

The seat-locking strategy

Design choice: pessimistic locking at the database, with an optimistic fast-path in Redis in front of it. Rationale below.

1. **Pre-generated seat rows.** When a show is scheduled, one `show_seat` row is inserted per (show, physical seat) — e.g. a 200-seat screen showing 3 times a day creates 600 rows, not 200. This turns “is seat 42 free for the 6pm show” into a single indexed row lookup instead of a derived computation.
2. **Redis fast path (optimistic, sub-millisecond).** A hold request first runs a Lua script that does `EXISTS + SET NX PX` across all requested seat keys atomically. If any seat is already locked, the whole request is rejected in Redis before MySQL is even touched — this is what lets a sold-out stampede fail fast instead of queueing on the database.
3. **MySQL conditional UPDATE (pessimistic, source of truth).**

```
UPDATE show_seat SET status='HELD', hold_id=?  
WHERE show_id=? AND seat_id IN (?,?,...) AND status='AVAILABLE';
```

The `UNIQUE(show_id, seat_id)` constraint plus this conditional `WHERE status='AVAILABLE'` means InnoDB takes a row lock per seat and the update only succeeds for rows still available. If `ROWS_AFFECTED != seat_count`, at least one seat was taken by a concurrent transaction, and the whole hold is rolled back — never a partial hold. **This is the real guarantee:** Redis being down, flushed, or skipped can never cause a double-booking, because this `UPDATE` still can't be satisfied twice for the same seat.

- Seat IDs are sorted before locking, so two requests wanting overlapping seat sets always attempt to lock them in the same order → no lock-ordering deadlocks.
 - Isolation level is `READ COMMITTED` (not the MySQL default `REPEATABLE READ`), specifically to avoid InnoDB gap locks on the `show_seat` range, which under `REPEATABLE READ` can cause spurious deadlocks between unrelated seat holds on the same show.
4. **Why pessimistic-at-the-DB rather than pure optimistic (version column + retry)?** Seat inventory is small and the hold window is short (10 min), so contention is bursty but the lock is held only for the duration of one short `UPDATE`, not the whole hold lifetime — an app-level optimistic-retry loop would just reimplement what the conditional `UPDATE` already gives us, with extra round-trips. Pure pessimistic (`SELECT ... FOR UPDATE` then application-side check) was avoided because it holds a row lock for the entire request handling time, including any Redis calls; the single conditional `UPDATE` is one round trip and holds the lock only for the transaction's brief duration.
 5. **Timed holds.** `seat_hold.expires_at` is the durable expiry. A background reaper (`main.py: reap_expired_holds`, polling every 5s) finds `ACTIVE` holds past `expires_at` using `FOR UPDATE SKIP LOCKED` (so the reaper never blocks on, or is blocked by, a live checkout), flips their seats back to `AVAILABLE`, and also clears the corresponding Redis keys — otherwise the Redis TTL (an independent, longer-lived safety net) would keep the seat un-holdable until it naturally expired.
 6. **Idempotent payment webhooks.** `payment_webhook_event.event_id` is the table's `PRIMARY KEY`. A duplicate webhook delivery (gateways routinely retry) hits a primary-key collision on `INSERT` and is caught and ignored — no double-processing of a payment, even under concurrent duplicate delivery.

Proof: load test results (this submission's own load test)

Run against the live service (loadtest/load_test.py), not simulated:

Scenario A — hot-seat stampede. 300 concurrent requests all racing for the *same 2 seats*:

```
requests=300 winners=1 conflicts=299 other=0
seats actually HELD in DB afterwards: 2 (must equal 2, never more)
RESULT: PASS - no double booking under concurrent stampede
```

Scenario B — realistic burst to sellout. 400 concurrent users, random small seat groups, each running the full hold → confirm → pay webhook chain, against a 24-seat show:

```
jobs=400 confirmed+paid=14 seat-conflicts=386 other=0
seats booked in DB: 24 / 24 capacity
duplicate-seat rows (must be empty): ()
RESULT: PASS - zero double-bookings across full show capacity
```

In both runs exactly the right number of seats end up booked, no seat is ever double-booked, and the losing requests fail fast with 409 Conflict rather than hanging or corrupting state.

P2 — Shows at a theatre on a given date

```
SET @theatre_id = 1;
SET @show_date = CURDATE();

SELECT
  t.name AS theatre,
  m.title AS movie,
  l.name AS language,
  f.name AS format,
  sc.name AS screen,
  ms.show_id,
  ms.show_date,
  TIME_FORMAT(ms.start_time, '%h:%i %p') AS show_time
FROM theatre t
JOIN screen sc ON sc.theatre_id = t.theatre_id
JOIN movie_show ms ON ms.screen_id = sc.screen_id
JOIN movie m ON m.movie_id = ms.movie_id
JOIN language l ON l.language_id = ms.language_id
JOIN show_format f ON f.format_id = ms.format_id
WHERE t.theatre_id = @theatre_id
      AND ms.show_date = @show_date
      AND ms.status = 'SCHEDULED'
ORDER BY m.title, ms.start_time;
```

This is served live at GET /theatres/{theatre_id}/shows?show_date=YYYY-MM-DD (see app/main.py).

Why it's fast at scale: movie_show carries UNIQUE(screen_id, show_date, start_time), so the join from screen (filtered by theatre_id) into movie_show (filtered by show_date) hits an index the whole way — confirmed with EXPLAIN in sql/03_p2_shows_by_date.sql, which also includes a grouped-by-movie variant (one row per movie with all its timings, matching the reference UI) and a live-seat-availability variant.

Repository layout

```

sql/
  01_schema.sql          -- P1: full DDL (3NF/BCNF), all FKs +
CHECK constraints
  02_seed.sql            -- sample cities/theatres/shows/seats
+ a few sample bookings
  03_p2_shows_by_date.sql -- P2: the show-listing queries +
EXPLAIN proof
app/
  main.py                -- FastAPI: hold / confirm / webhook /
P2 endpoints + reaper
  db.py                  -- MySQL connection pool + deadlock-
retrying transaction runner
  seatlock.py            -- Redis Lua-script seat locks (fast
path)
  config.py              -- tunables (hold TTL, pool size,
etc.)
loadtest/
  load_test.py           -- the load test referenced above
(hot-seat + realistic-burst)
docs/
  submission.md / .pdf   -- this document
README.md                -- how to run everything locally

```